

Assignment 3: Autonomous Multi-Doc Financial Analyst Report

1. 比較不同的 Embedding Models (20%)

在本實驗中，測試並比較了不同的 Embedding 模型來將 PDF 文件向量化。

- **Model A (Baseline):** `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`
 - **特色：**這是一個輕量級的開源多語系模型，適合建立基本的語意搜尋。
 - **實驗觀察：**由於模型較小且專注於一般語意，在處理結構極度複雜的財務報表（10-K）時，對「縮寫、財報特殊術語（如 Cost of sales - Services）」的敏感度較低。檢索時容易給出擁有部分相符字詞、但年份或確切科目錯誤的雜訊段落。
- **Model B (Comparison):** `BAAI/bge-base-en-v1.5`
 - **特色：**架構較大且參數訓練更深，對長文本與複雜專業術語的理解能力較強。
 - **實驗觀察：**當替換為進階模型後，針對如 "R&D" 或 "Gross Margin" 的複雜查詢，檢索到的 Chunks 精準度明顯提升，減少了 Grader 節點觸發 `no` 並被迫重寫指令的次數，使得最終的回答 (Final Answer) 更加精確。

📋 實測 Log 證據：

```
=====
📄 ASSIGNMENT 3 EVALUATION REPORT
🤖 Agent Mode: GRAPH
🔗 Embedding Model: [ADVANCED] BAAI/bge-base-en-v1.5
=====
📊 FINAL SCORE: 8/14
```

2. 比較 LangGraph 與 LangChain (30%)

在本作業中，我們實作了傳統的線性 ReAct Agent (Task A) 以及基於狀態機的 LangGraph Agent (Task B-E)，兩者在架構與執行表現上有明顯的優劣差異：

- **架構與思維模式 (Architecture)**
 - **LangChain (ReAct):** 採用黑箱式的 `Thought -> Action -> Observation` 循環。開發者難以介入中間過程，全依賴 LLM 的自主決策。
 - **LangGraph:** 採用有向圖 (Graph) 結構。將每個步驟拆解為實體節點 (`Router`、`Grader`、`Generator`、`Rewriter`)，並透過明確的條件邊界 (Conditional Edges) 控制流程。
- **穩定度與除錯 (Stability & Debugging)**

- **LangChain:** 非常不穩定。在此次測試中，經常發生 `OUTPUT_PARSING_FAILURE` (格式解析失敗) 的錯誤，因為 LLM 容易過度生成或忽視格式規範。此外，當找不到答案時容易輕易放棄 (回答 `I don't know`)。
- **LangGraph:** 穩定且可控。我們可以清晰看見資料流經哪個節點。如果 `Grader` 判斷資料無用，控制權會交由 `Rewriter` 重新組織提問並再次檢索，完美展現了 **自我糾錯 (Self-Correction)** 的特性。

- **Token 消耗與 API 限制 (Token Efficiency)**

- **LangChain:** 最致命的缺點在於它會在 `agent_scratchpad` 中保留每一次檢索到的冗長財報文字。當搜尋多次後，累積的輸入 Token 長度會呈現指數成長，輕易觸發 API 的 Rate Limit (如 `429 Too Many Requests` 及 Input Token Count 限制)。
- **LangGraph:** 每次節點傳遞只交換必要的狀態 (State)，不用把歷史的大型表格無腦疊加給下一次檢索使用，大幅節省了 Token 成本並提高了檢測速度。

📋 實測 Log 證據對比：Legacy Agent (卡死、報錯與低分)

```
🤖 Agent Mode: LEGACY
> Entering new AgentExecutor chain...
I need to find out who signed the 10-K report...
Action: search_tesla_financials
Action Input: "who signed the 10-K report"
... (Infinite Iterations & Parsing Error) ...
📊 FINAL SCORE: 5/14
```

Graph Agent (自我糾錯防護網)

```
🤖 Agent Mode: GRAPH
--- 🔍 RETRIEVING ---
--- 📊 GRADING ---
Relevance Grade: no (Raw: no)
--- 📝 REWRITING QUERY ---
New Question: What were Tesla's Research and Development (R&D) expenses...
--- 🔍 RETRIEVING ---
```

3. Chunk Size 的影響與權衡關係 (20%)

在處理如資產負債表 (Balance Sheet) 和損益表 (Income Statement) 等大型結構化表格時，文件切割大小 (`chunk_size`) 對檢索表現有決定性的影響。我們預設使用了 `chunk_size=2000`，以下探討調小與調大所帶來的權衡：

- **調小 Chunk Size (例如 `chunk_size=500` 或更低)**

- **優勢 (Context Precision 高)：** 檢索結果非常集中，幾乎不包含無關雜訊，Token 消耗量極低。
- **劣勢：** 對於大型財報表格是**災難性的**。因為表格通常很寬很長，如果 Chunk 太小，極容易將「表頭的年份 (如 2024)」與「底下的數值 (如 391,035)」硬生生切斷。這會導致 LLM 檢索到了數字，卻

不知道這數字屬於哪一年，最終回報 **I don't know** 或抓錯年份 (例如錯抓 2023 年資料)。

- 調大 **Chunk Size** (例如 **chunk_size=4000** 或更高)
 - **優勢 (Context Completeness 高)**：完整保存了表格的上下文與階層結構。表頭的年份、註解與具體數值會被包裝在同一個 **Chunk** 中，LLM 能夠順利進行縱向與橫向的對齊。
 - **劣勢**：犧牲了精確度。一個極大的 **Chunk** 包含了大量與使用者問題無關的業務描述 (**Noise**)。這不僅會急劇增加 **Input Token** 的花費，還有可能引發大語言模型的「迷失在中間 (**Lost in the Middle**)」效應，導致重要資訊被忽略。

📖 實測 Log 證據 (**chunk_size=500** 極端測試)：

```
--- 📄 REWRITING QUERY ---  
New Question: What were Tesla's capital expenditures for the fiscal year 2024,  
disaggregated by major asset class...  
--- 🔍 RETRIEVING ---  
Routed to: tesla  
--- 📊 GRADING ---  
Relevance Grade: no (Raw: no)  
(Max retries reached, generating anyway...)
```

(從 Log 中可見，由於表格被切得太碎，缺失上下文，導致 **Grader** 不斷給出 **no** 並觸發重試，最終仍然找不到完整答案)

總結：**chunk_size=2000** 是一個在「避免切斷表頭上下文」與「維持合理的 **Token** 花費」之間取得平衡的設定點。但在面對高度格式化的財報時，最佳解或許不是單純調整字數，而是引入針對表格優化的解析器或階層式切割 (**Hierarchical Chunking**)。